这段文字描述的是著名的 **KM 算法（Kuhn-Munkres Algorithm）**，它是解决**二分图最大权完美匹配**问题的经典算法。

为了让你听清楚这个原理，我们可以把这个算法拆解为“核心思想”、“核心工具”和“运行流程”三个部分：

1. 核心思想：从“可行”到“最优”

KM 算法巧妙地利用了对偶问题的思想。它不直接去找最大权，而是通过维护一组**“顶标”（Labels），构造一个**“相等子图”**。

- **定理背景：** 如果我们在一个二分图中，只利用那些满足 $lx(i) + ly(j) = w(i, j)$ 的边进行匹配，并最终找到了一个**完美匹配**，那么这个匹配就是原图的**最大权匹配**。
- **直观理解：** 顶标可以看作是每个点的“期望值”。
  - $lx(i)$ 是左边点的期望，$ly(j)$ 是右边点的期望。
  - 算法要求始终满足 $lx(i) + ly(j) \geq w(i, j)$（即：总期望不能低于实际收益）。
  - 当期望之和刚好等于收益时（$lx + ly = w$），这条边就是“理想”的，我们把它加入相等子图。

2. 核心工具：相等子图与交错树

- **相等子图 (Equality Subgraph)：** 只有满足 $lx(i) + ly(j) = w(i, j)$ 的边才在这个图里。在这个图里找匹配，就是在找“最划算”的组合。
- **增广路与交错树：**
  - 我们像求普通最大匹配（匈牙利算法）一样，在相等子图中找增广路。
  - 如果找到了，匹配数加 1。
  - 如果找不到，就会形成一棵**交错树**。这意味着在目前的“期望”下，我们无法完成更多的匹配。

3. 原理的核心：如何“调整”顶标？

当我们在相等子图中找不到增广路时，说明当前的“门槛”设得太高了，需要降低期望，引入新的边。

图片中提到的 S, T, S', T' 就是为了找到调整的幅度 Δ：

1. **找出差距 Δ：** 我们观察所有“左端点在交错树中 (S)，右端点不在交错树中 (T')”的边。
   - 计算这些边的差距：$d = lx(u) + ly(v) - w(u, v)$。
   - 取这些差距中的最小值作为 Δ：$\Delta = \min\{lx(u) + ly(v) - w(u, v) \mid u \in S, v \notin T\}$。
2. **更新顶标：**
   - **对于在交错树里的左点 (S)**：期望降低，即 $lx(i) = lx(i) - \Delta$。
   - **对于在交错树里的右点 (T)**：由于它们已经匹配了，为了维持原有的相等关系，期望要补偿回来，即 $ly(j) = ly(j) + \Delta$。
3. **结果：**

- 原本在树内的边，一减一加，和不变，依然在相等子图中。
- **关键：** 至少会有一条原本不在相等子图中的边（那些产生最小 Δ 的边），因为左边顶标降了 Δ，现在满足了 $lx + ly = w$，从而**进入了相等子图**。
- 这样，我们就扩大了搜索范围，可以继续找增广路了。

总结一下流程：

1. **初始化：** 给左边点定最高期望（相连边的最大权），右边定为 0。
2. **试匹配：** 在相等子图中找增广路。
3. **遇阻调整：** 找不到增广路时，通过计算 Δ 调整顶标，让新的边进入相等子图。
4. **循环：** 重复上述过程，直到每个点都匹配上（达到完美匹配）。

一句话总结其妙处： KM 算法通过不断微调每个点的“期望值”，在保证不违背权值上限的前提下，逐步把那些权值较大的边拉进匹配范围，直到拼凑出一个完整的完美匹配。